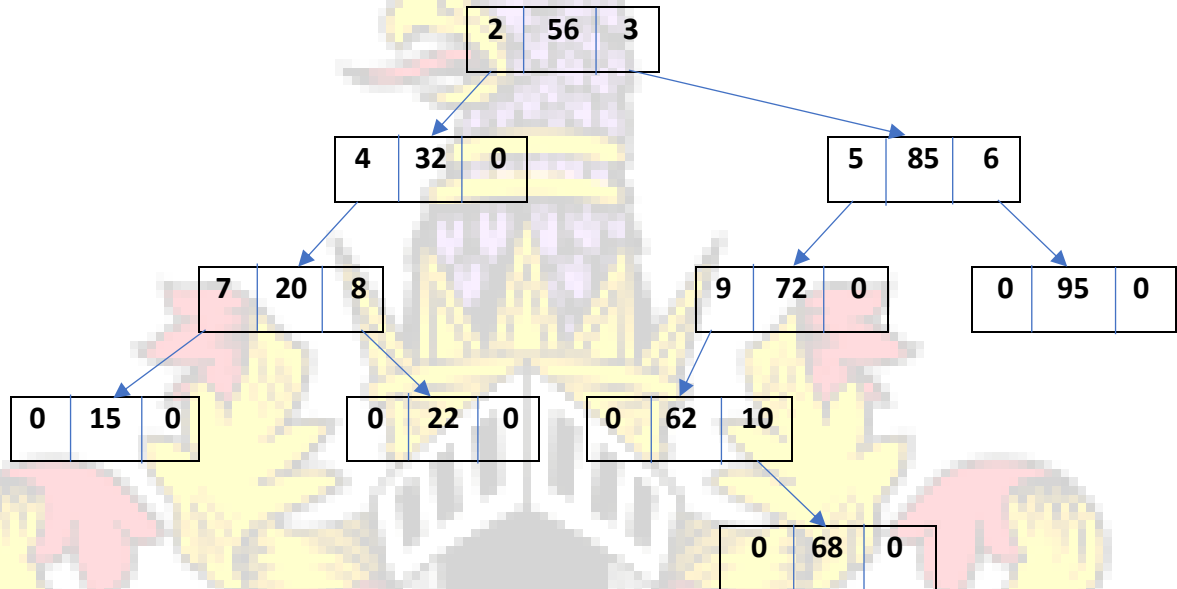




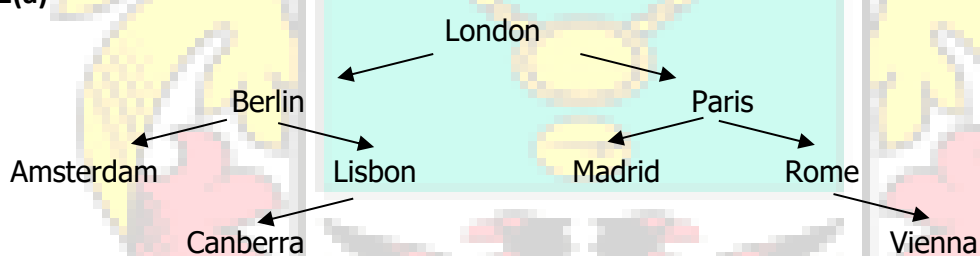
Tutorial S1H: Binary Search Tree (Solutions)

1(a)



- 1(b) (i) 56, 32, 20, 15, 22, 85, 72, 62, 68, 95
(ii) 15, 20, 22, 32, 56, 62, 68, 72, 85, 95
(iii) 15, 22, 20, 32, 68, 62, 72, 95, 85, 56

2(a)



2(b) Using 3 1-dimensional arrays. Left and Right: array of integers. Data: array of strings.

Index
1
2
3
4
5
6
7
8
9

Left
4
6
0
9
7
0
0
0
0

Data
London
Paris
Rome
Berlin
Lisbon
Madrid
Canberra
Vienna
Amsterdam

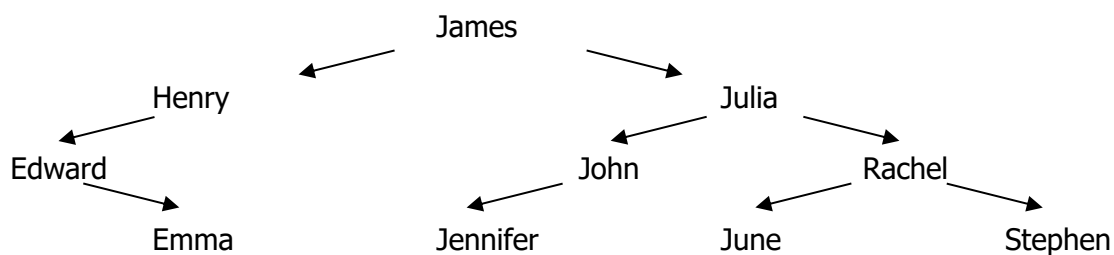
Right
2
3
8
5
0
0
0
0
0

2(c) London, Paris

2(d)

```
PROCEDURE PrintAscending(index) // index is root node of
                                // tree (or subtree)
    IF index is not NULL THEN
        PrintAscending(Tree[index].getLeftPointer())
        Output Tree[index].getRootData()
        PrintAscending(Tree[index].getRightPointer())
    ENDIF
ENDPROCEDURE
```

3



(b)

```
PROCEDURE TreeInsert (Tree, NewNode)
    Prev ← NULL
    Cur ← Tree.Root
    WHILE Cur <> NULL
        Prev ← Cur
        IF (NewNode.key < Cur.key) THEN
            Cur ← Cur.GetLeftPointer()
        ELSE
            Cur ← Cur.GetRightPointer()
        ENDIF
    ENDWHILE
    IF Prev = NULL THEN
        Tree.Root ← NewNode
    ELSE
        IF NewNode.key < Prev.key THEN
            Prev.SetLeftPointer(NewNode)
        ELSE
            Prev.SetRightPointer(NewNode)
        ENDIF
    ENDIF
ENDPROCEDURE
```

- (c) Pre-order: James, Henry, Edward, Emma, Julia, John, Jennifer, Rachel, June, Stephen
In-order: Edward, Emma, Henry, James, Jennifer, John, Julia, June, Rachel, Stephen
Post-order: Emma, Edward, Henry, Jennifer, John, June, Stephen, Rachel, Julia, James

(d) Either **John** replaces Julia (John is largest node in left subtree), or **June** replaces Julia (June is smallest node in right subtree).

(e)(i)

A tree node:

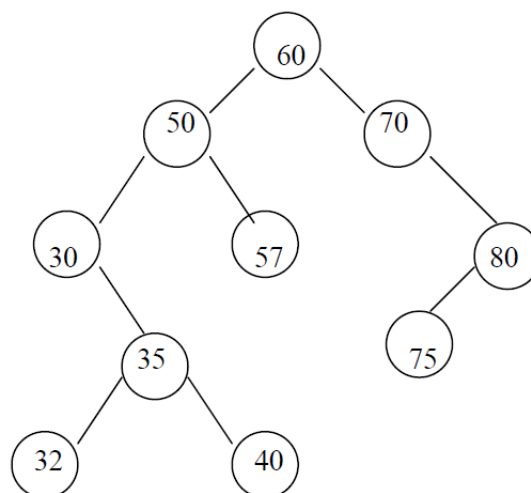
Node
Private attribute: Name : STRING LeftPointer : Pointer (INTEGER) RightPointer : Pointer (INTEGER)
Public Methods:
GetName() GetLeftPointer() GetRightPointer() SetName(name: STRING) SetLeftPointer(ptr: INTEGER) SetRightPointer(ptr: INTEGER)

A binary Tree ADT:

BinaryTree
Private attribute: RootPtr : Pointer (INTEGER)
Public Methods:
AddNode (name: STRING) InOrder (ptr: INTEGER) PreOrder (ptr: INTEGER)

(e)(ii) ADT operations: AddNode, InOrder, PreOrder

4(a) The first value to be inserted is put into the root node. Thereafter, each value to be inserted begins by comparing itself to the value in the root node, moving left if it is less, or moving right if it is greater. This continues at each level until it can be inserted as a new leaf node.



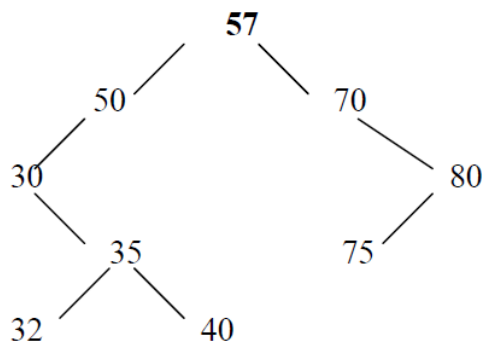
(b)(i)

```
FUNCTION Search(x, item) //x points to root node of tree(or subtree)
  IF (x = NULL)
    RETURN FALSE
  ELSE IF (item < x->data) THEN
    Search(x->left, item) // recursive search left subtree
  ELSE IF (item > x->data)
    Search(x->right, item) // recursive search right subtree
  ELSE
    RETURN TRUE // item = x->data
  RETURN TRUE
ENDFUNCTION
```

(b)(ii)

```
PROCEDURE SortedDisplay(x) //x points to root node of tree(or subtree)
  IF (x IS NOT NULL)
    SortedDisplay(x->left)
    OUTPUT x->data
    SortedDisplay(x->right)
  ENDIF
ENDPROCEDURE
```

(c) Promote 57: largest value in left subtree



Promote 70: smallest value in right subtree

